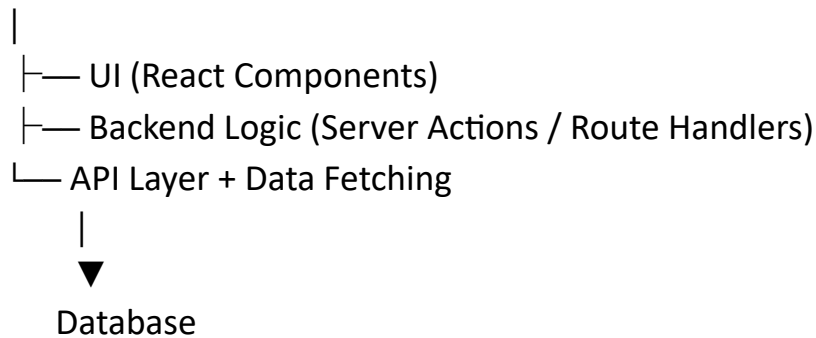


NEXT JS

Next.js App



Everything lives **inside one project**.

That's why it's called **full-stack**.

API routing

Mapping a URL endpoint to backend code.

Example request:

[GET /api/users](#)

Next.js automatically looks for a file that matches that route.

In the **App Router**, the file must be:

`app/api/users/route.ts`

So the **folder structure = route structure**.

```
app
├── api
│   └── users
│       └── route.ts
```

URL becomes:

`/api/users`

No router setup needed.

Route handlers

Route Handlers are the way to create **backend API endpoints**.

Think of them as the **Next.js version of Express routes**.

Next.js Route Handler

app/api/users/route.ts

```
export async function GET() {  
  return Response.json({ users: [] })  
}
```

Endpoint becomes:

/api/users

2. Where Route Handlers Live

In the **App Router**, API routes must be inside:

app/api/

Example structure:

```
app  
├─ api  
│   └─ users  
│       └─ route.ts  
│   └─ posts  
│       └─ route.ts
```

Routes become:

/api/users

/api/posts

3. HTTP Methods

Instead of writing `app.get`, `app.post`, etc., you **export functions**.

```
export async function GET() {}
export async function POST() {}
export async function PUT() {}
export async function DELETE() {}
export async function PATCH() {}
```

4. Request Object

Route handlers receive a **Request object** (Web API standard).

```
export async function POST(request: Request) {
  const body = await request.json()

  return Response.json(body)
}
```

Example request body:

```
{
  "name": "Prajwal"
}
```

5. Dynamic Routes

Just like pages.

Folder name uses `[ ]`.

`app/api/users/[id]/route.ts`

Example:

```
export async function GET(
  request: Request,
  { params }: { params: { id: string } }
) {
  return Response.json({ id: params.id })
}
```

Request:

/api/users/42

Response:

```
{ id: "42" }
```

6. Query Parameters

Example request:

/api/users?role=admin

Code:

```
export async function GET(request: Request) {  
  const { searchParams } = new URL(request.url)  
  
  const role = searchParams.get("role")  
  
  return Response.json({ role })  
}
```

The Big Auth Flow

Typical login flow:

1. User submits login form
2. Server verifies credentials
3. Server creates token/session
4. Server stores token in cookie
5. Browser sends cookie automatically on future requests
6. Server reads cookie → identifies user

So the important pieces are:

Cookies → store auth token

Headers → send token (optional)

Route handlers → verify token

2. Login API (Creating Cookie)

Route handler:

app/api/login/route.ts

Example:

```
import { cookies } from "next/headers"
```

```
export async function POST(request: Request) {  
  const body = await request.json()
```

```
  const { email, password } = body
```

```
  // pretend we validated user  
  const token = "abc123"
```

```
  cookies().set("token", token, {  
    httpOnly: true,  
    secure: true,
```

```
    path: "/",  
  })  
  
  return Response.json({ message: "logged in" })  
}
```

Important part:

`cookies().set()`

This sends a Set-Cookie header to the browser.

What the Browser Receives

Response header:

Set-Cookie: token=abc123; HttpOnly; Secure

Now the browser stores it.

4. Browser Automatically Sends Cookie

Next request:

GET /api/profile

Browser automatically sends:

Cookie: token=abc123

You don't manually attach it.

5. Reading Cookie in Route Handler

app/api/profile/route.ts

```
import { cookies } from "next/headers"
```

```
export async function GET() {  
  const cookieStore = cookies()
```

```
const token = cookieStore.get("token")

if (!token) {
  return Response.json(
    { error: "Unauthorized" },
    { status: 401 }
  )
}

return Response.json({
  message: "user authenticated",
  token: token.value
})
}
```

7. Reading Request Headers

So the mental model:

```
request.headers.get("header-name")
```

```
export async function GET(request: Request) {
  const userAgent = request.headers.get("user-agent")

  return Response.json({
    agent: userAgent
  })
}
```

Example request header:

User-Agent: Mozilla/5.0

Response:

```
{"agent": "Mozilla/5.0"}
```

8. Cookies vs Headers (Important Interview Question)

Cookies

browser sends automatically
good for web apps
supports httpOnly security
used in session auth

Headers

client sends manually
good for APIs/mobile
often used with JWT
used in token auth

Route and Page Protection

Method 1 — Check Auth in the Page (Server Component)

Since pages run on the server by default, you can read cookies directly.

Example protected page:

`app/dashboard/page.tsx`

```
import { cookies } from "next/headers"
import { redirect } from "next/navigation"
```

```
export default function Dashboard() {
  const token = cookies().get("token")
```

```
  if (!token) {
    redirect("/login")
  }
```

```
  return <h1>Dashboard</h1>
}
```


Flow:

User → dashboard

|

Read cookie

|

No cookie → redirect

✓ Simple

✗ You must repeat this on every page

Method 2 — Middleware (Best for Large Apps)

This protects **entire route groups** automatically.

Create:

middleware.ts

Example:

```
import { NextResponse } from "next/server"
import type { NextRequest } from "next/server"

export function middleware(request: NextRequest) {
  const token = request.cookies.get("token")

  if (!token) {
    return NextResponse.redirect(new URL("/login", request.url))
  }

  return NextResponse.next()
}
```

Now configure what routes to protect:

```
export const config = {
  matcher: ["/dashboard/:path*"]
}
```

Meaning:

/dashboard
/dashboard/settings
/dashboard/profile

All require auth.

This runs **before the page loads**.

4. Method 3 — Protect API Routes

Your API routes should also check auth.

Example:

[app/api/profile/route.ts](#)

```
import { cookies } from "next/headers"

export async function GET() {
  const token = cookies().get("token")

  if (!token) {
    return Response.json(
      { error: "Unauthorized" },
      { status: 401 }
    )
  }

  return Response.json({ user: "Prajwal" })
}
```

So both **pages and APIs stay protected**.

Data fetching

In Next.js you can fetch data **on the server or on the client**.

1. The Simplest Example

In a **Server Component** (default in Next.js):

```
export default async function Page() {  
  const res = await fetch("https://api.example.com/users")  
  const users = await res.json()  
  
  return <div>{users.length}</div>  
}
```

What happens:

Browser → request page

Server → fetch data

Server → render HTML

Browser → receives ready page

This is **server-side data fetching**.

2. Why This Is Powerful

In plain **React** you usually do:

```
useEffect(() => {  
  fetch(...)  
}, [])
```

Meaning:

Page loads

↓

JS runs

↓

API request happens

↓

UI updates

User waits longer.

With **Next.js server fetching**:

Server fetches data first

↓

HTML already contains data

↓

Page loads faster

Better for:

- SEO
- performance
- security

3. Fetching From Your Own API

Example:

```
export default async function Page() {  
  const res = await fetch("http://localhost:3000/api/users")  
  const users = await res.json()  
  
  return <pre>{JSON.stringify(users)}</pre>  
}
```

But in real apps you usually **call the database directly**, not your API.

4. Fetching From Database (Better)

Example with **Prisma**:

```
import { prisma } from "@/lib/prisma"  
  
export default async function Page() {  
  const users = await prisma.user.findMany()
```

```
    return <div>{users.length}</div>
  }
}
```

Flow:

Page request



Server component



Database query



HTML sent to browser

No API needed.

5. Client-Side Fetching

Sometimes you need client-side fetching.

Example:

"use client"

```
import { useEffect, useState } from "react"
```

```
export default function Page() {
  const [users, setUsers] = useState([])
```

```
  useEffect(() => {
    fetch("/api/users")
      .then(res => res.json())
      .then(setUsers)
  }, [])
```

```
  return <div>{users.length}</div>
}
```

Use this when:

- UI updates frequently
- real-time data
- user-triggered actions

7. Three Data Fetching Modes

Mode	Meaning
Static	data fetched at build
Dynamic	fetched on request
Revalidated	cached then refreshed

Next.js decides automatically.

In Next.js, data fetching can happen in server components, client components, or route handlers, but the preferred approach is server-side fetching to improve performance and SEO.

MiddleWares

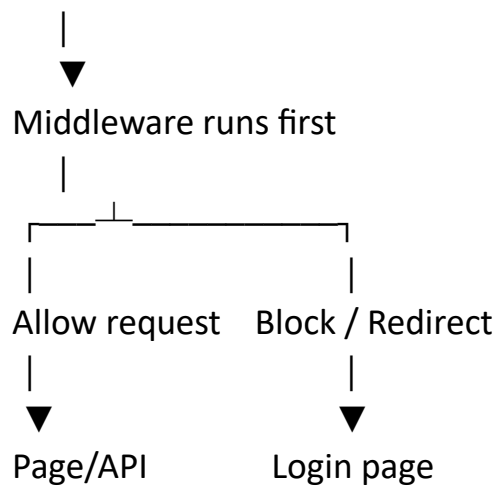
middleware in Next.js is basically **code that runs *before* a request reaches your page or API.**

Think of it like a **gatekeeper**.

1. Simple Mental Model

When a user requests a page:

User Request



So middleware is perfect for:

- authentication
- redirects
- logging
- rate limiting
- geo-based routing

2. Where Middleware Lives

You create a file in the **root of your project**:

middleware.ts

Example project:

app
middleware.ts
next.config.js

3. Basic Middleware Example

```
import { NextResponse } from "next/server"
import type { NextRequest } from "next/server"

export function middleware(request: NextRequest) {
  console.log("middleware running")

  return NextResponse.next()
}
```

Meaning:

Request → middleware → continue to page

4. Authentication Middleware (Most Common Use)

Example:

```
import { NextResponse } from "next/server"
import type { NextRequest } from "next/server"

export function middleware(request: NextRequest) {
  const token = request.cookies.get("token")

  if (!token) {
    return NextResponse.redirect(
      new URL("/login", request.url)
    )
  }

  return NextResponse.next()
}
```


5. Protecting Specific Routes

You usually don't want middleware to run everywhere.

Use **matcher config**.

```
export const config = {  
  matcher: ["/dashboard/:path*"]  
}
```

Meaning:

/dashboard

/dashboard/settings

/dashboard/profile

All protected.

Important Detail (Interview Favorite)

Next.js middleware runs on the **Edge Runtime**, not a full **Node.js** environment.

Meaning:

 You **cannot use**:

fs

net

heavy node libraries

But you **can use**:

cookies

headers

URL

fetch

To run middleware BEFORE ROUTE

middleware.ts

and inside it:

```
export const config = {  
  matcher: ["/dashboard/:path*"]  
}
```

Matcher Examples (very useful)

Protect one route

```
matcher: ["/api/users"]
```

Protect all API routes

```
matcher: ["/api/:path*"]
```

Protect dashboard pages

```
matcher: ["/dashboard/:path*"]
```

Multiple routes

```
matcher: ["/dashboard/:path*", "/api/private/:path*"]
```

Sometimes interviewers ask:

Can we put middleware inside app/api/users?

Answer:

 No

 Middleware **must be in the root** (middleware.ts)

Next.js automatically runs it based on the matcher.

Edge functions

Edge Functions in **Next.js** run your backend code on **servers located close to the user** instead of a single central server.

This reduces **latency** and makes responses **much faster**.

Key Points

- Run on a global edge network (like CDN servers)
- Execute **closer to the user**
- Used for **fast request processing**
- Often used with **middleware and lightweight APIs**

Common Use Cases

- Authentication checks
- Redirects
- A/B testing
- Geo-based routing
- Rate limiting

Limitations

Edge runtime is **not full Node.js**, so things like:

- fs
- heavy Node libraries

may not work.

1. What are Route Handlers in Next.js?

Answer

Route handlers are backend API endpoints created inside the `app/api` directory. They handle HTTP requests by exporting functions like GET, POST, PUT, or DELETE.

2. How is API routing implemented in Next.js?

Answer

Next.js uses **file-based API routing**.

The folder structure inside `app/api` automatically defines API endpoints.

Example:

`app/api/users/route.ts`

What is the difference between Middleware and Route Handlers?

Answer

Middleware	Route Handler
Runs before request	Handles the request
Used for auth/redirects	Used for API logic
Runs globally	Runs per route

Why is server-side data fetching preferred?

Answer

Because it improves:

- performance
- SEO
- security

The server fetches data before sending HTML to the browser.

What is the request lifecycle in Next.js?

Answer

Typical request flow:

User request



Middleware



Route Handler / Server Component



Data Fetching



Response sent to browser

Difference between Cookies and Authorization Headers?

Answer

Cookies	Headers
Sent automatically by browser	Sent manually by client
Good for web sessions	Good for APIs/mobile
Supports httpOnly security	Often used with JWT

Why does Next.js use Web Request/Response APIs instead of Express req/res?

Answer

Next.js uses **Web standard APIs** so the same code can run in:

- Node runtime
- Edge runtime
- browsers

This improves portability and performance.

Q: Why use API routes if server components can access the database directly?

Answer

API routes are still needed for:

- mobile clients
- third-party integrations
- webhooks
- external services

Why does Next.js prefer Server Components by default?

Answer

Server Components allow data fetching and rendering on the server, reducing the JavaScript sent to the browser and improving performance and SEO.

Benefits:

- smaller client bundle
- secure server-side logic
- direct database access

When should you NOT use Server Components?

Answer

When the component needs:

- browser APIs (window, localStorage)
- React hooks like useState or useEffect
- event handlers

Then you must use:

"use client"

Why is calling your own API from Server Components considered an anti-pattern?

Answer

Because the server already has direct access to the database or backend logic. Calling your own API adds **unnecessary network overhead**.

Why does Middleware run on Edge Runtime?

Answer

Middleware runs on the edge to **reduce latency** and process requests closer to the user.

This allows faster authentication checks and routing decisions.

How does Next.js decide if a page is static or dynamic?

Answer

Next.js analyzes data usage.

Static if:

- no dynamic data
- cached fetch requests

Dynamic if:

- cookies()
- headers()
- cache: "no-store"

Why does using cookies() make a page dynamic?

Answer

Because cookies are request-specific.

Each request may have different cookies, so the page must render **per request**, preventing static generation.

What problem does Streaming solve in Next.js?

Answer

Streaming allows sending **partial HTML to the browser while the rest loads**.

This improves perceived performance.

When should you use Route Handlers instead of Server Actions?

Answer

Route handlers are needed when:

- building public APIs
- supporting mobile apps
- handling webhooks
- integrating third-party services

Server actions are mostly for **internal UI interactions**.

Why are cookies preferred over localStorage for authentication?

Answer

Cookies can be:

- httpOnly
- secure
- sameSite

This prevents JavaScript access and reduces XSS attack risks.

Why is Next.js considered a full-stack framework?

Because it provides both:

Frontend:

- React UI
- routing
- rendering

Backend:

- API routes
- server actions
- middleware
- data fetching

All inside **one project**.